FedSRD: Sparsify-Reconstruct-Decompose for Communication-Efficient Federated Large Language Models Fine-Tuning

Guochen Yan
Beijing Key Laboratory of Data Intelligence and Security, National Engineering Research Center for Software Engineering, School of Computer Science, Peking University
Beijing, China
2301111972@stu.pku.edu.cn

Luyuan Xie
Beijing Key Laboratory of Data Intelligence and Security, School of Software and Microelectronics, Peking University
Beijing, China
2201110745@stu.pku.edu.cn

Qingni Shen*
Beijing Key Laboratory of Data Intelligence and Security, School of Software and Microelectronics, Peking University
Beijing, China
qingnishen@ss.pku.edu.cn

Yuejian Fang
Beijing Key Laboratory of Data Intelligence and Security, School of Software and Microelectronics, Peking University
Beijing, China
fangyj@ss.pku.edu.cn

Zhonghai Wu*
Beijing Key Laboratory of Data Intelligence and Security, National Engineering Research Center for Software Engineering, Peking University
Beijing, China
wuzh@pku.edu.cn

Abstract

The current paradigm of training large language models (LLMs) on public available Web data is becoming unsustainable as high-quality data sources in specialized domains near exhaustion. Federated Learning (FL) emerges as a practical solution for the next generation of AI on a decentralized Web, enabling privacy-preserving collaborative fine-tuning on decentralized private data. While Low-Rank Adaptation (LoRA) is standard for efficient fine-tuning, its federated application faces a critical bottleneck: communication overhead under heterogeneous network conditions. Structural redundancy in LoRA parameters increases communication costs and causes aggregation conflicts. To address this, we propose FedSRD, a Sparsify-Reconstruct-Decompose framework for communication-efficient federated LLM fine-tuning. We introduce importance-aware sparsification to reduce the upload parameter count while preserving the structural integrity of LoRA updates. The server aggregates updates in full-rank space to mitigate conflicts, then decomposes the global update into a sparse low-rank format for broadcast, ensuring a symmetrically efficient cycle. We also propose an efficient variant, FedSRD-e, to reduce computational overhead. Experiments on 10 benchmarks show our framework significantly reduces communication costs by up to 90% while improving performance on heterogeneous client data.

*Corresponding author

This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792144

CCS Concepts

• **Computing methodologies → Cooperation and coordination**.

Keywords

Federated Learning; Large Language Models; Communication; LoRA

ACM Reference Format:
Guochen Yan, Luyuan Xie, Qingni Shen, Yuejian Fang, and Zhonghai Wu. 2026. FedSRD: Sparsify-Reconstruct-Decompose for Communication-Efficient Federated Large Language Models Fine-Tuning. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3774904.3792144

1 Introduction

Large Language Models (LLMs), pre-trained on vast web-scale corpora, have demonstrated remarkable generalist capabilities [17, 30]. However, realizing their full potential for specialized downstream applications requires fine-tuning on diverse, high-quality, domain-specific data. Such data is often proprietary and distributed across numerous private sources, resulting in fragmented "data silos". As centralizing this sensitive data is frequently impractical due to privacy regulations and cost constraints, Federated Learning (FL) offers a compelling solution, enabling multiple clients to collaboratively fine-tune a global shared LLM without direct data exchange. This paradigm is central to building the next generation of intelligent, decentralized Web applications [34, 40].

While FL provides a privacy-preserving solution, fine-tuning billion-parameter LLMs across distributed clients remains computationally and communicatively prohibitive. To overcome this,

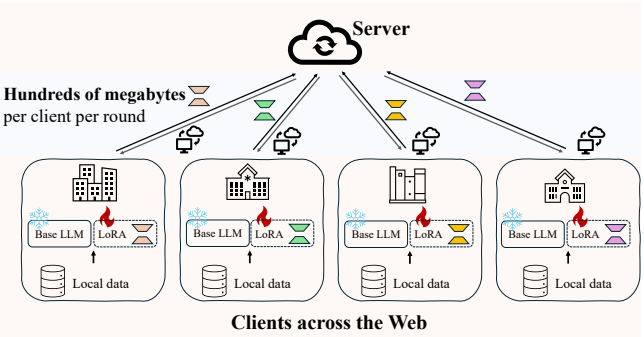

Figure 1: An illustration of Federated LoRA Fine-tuning. Clients train and communicate only the LoRA matrices. However, this payload can still amount to hundreds of megabytes per round, posing a significant bottleneck.

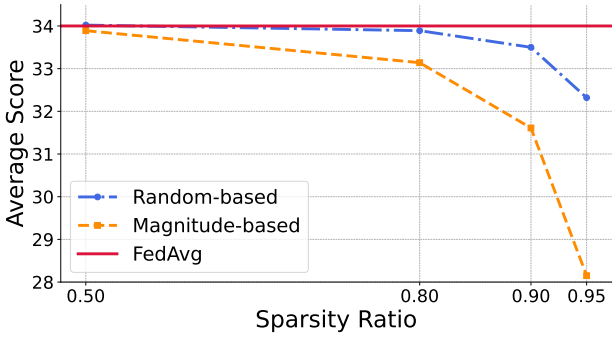

Figure 2: Performance comparison under different sparsity ratios. Both random and magnitude-based sparsification suffer performance decline at high sparsity ratios, motivating the need for a more advanced, structure-aware strategy.

Parameter-Efficient Fine-Tuning (PEFT) methods, particularly Low-Rank Adaptation (LoRA) [15], have become the standard [7, 19, 44]. As illustrated in Figure 1, LoRA significantly reduces the training burden by optimizing only a small set of trainable, low-rank matrices. In a federated setting, clients train and communicate only these LoRA parameters, achieving performance comparable to full-parameter fine-tuning at a fraction of the resource cost.

Despite the efficiency, the iterative communication of LoRA parameters remains a substantial overhead. This becomes a critical bottleneck when deployed across the heterogeneous and unpredictable network conditions characteristic of the global Web and mobile systems [8, 28]. High communication costs can exclude clients with limited bandwidth, such as those in rural areas or on mobile devices, undermining data diversity and posing fairness and inclusivity challenges for the global Web ecosystem. While sparsification is a common strategy for reducing communication, conventional methods are ill-adapted for federated LoRA fine-tuning on non-IID data. As shown in Figure 2, naïve approaches like magnitude-based pruning [1] disrupt the structural relationship between LoRA matrices A and B, while random sparsification [45] also degrades at high sparsity ratios. Crucially, both methods fail to preserve client-specific knowledge and exacerbate conflicts during server-side aggregation under data heterogeneity. This results in an undesirable trade-off: reduced communication at the expense of significant performance loss. This leads to our central research question:

How can we significantly reduce communication costs in federated LLMs fine-tuning without sacrificing model performance under heterogeneous client data?

To address this question, we introduce FedSRD, a novel Sparsify-Reconstruct-Decompose framework for communication-efficient federated LLMs fine-tuning. Our framework ensures high model performance through three sequential operations.

First, on the client side, we develop an importance-aware **Sparsification** strategy. Instead of relying on parameter magnitude, we calculate the importance scores that quantify the contribution of each element in update matrices (ΔB and ΔA) to the overall update (ΔW). This effectively preserves the critical structural interplay between LoRA matrices. Furthermore, recognizing that different layers serve distinct functions, we adaptively determine layer-wise sparsity ratios based on the kurtosis of its importance scores. Aggressive pruning is applied where high importance is concentrated. Second, the server executes the **Reconstruction** and **Decomposition**. Upon receiving sparse updates, the server first reconstructs each client's full-rank update (ΔW) before aggregation. This enables aggregation to occur in the full-rank space, which is mathematically sound and effectively mitigates the update conflicts from non-IID data [37, 39]. Finally, to create a symmetrically efficient communication cycle, the server performs a novel **Decomposition** of the aggregated global update back into a sparse, low-rank format for clients to download. Using a Taylor expansion-based approximation, we employ an alternating scheme that solves for and transmits only one sparse low-rank matrix per round. Furthermore, We propose FedSRD-e, a computational-efficient variant that bypasses intensive SVD operations for resource-constrained servers.

We demonstrate the effectiveness of FedSRD through extensive experiments that simulate practical non-IID scenarios by finetuning an LLM with clients from diverse specialist domains. Our evaluation on 8 in-domain and 2 out-of-domain benchmarks shows that FedSRD reduces both upload and download communication by 90% while achieving even better performance. In summary, our contributions are:

- We propose a novel, structure-aware sparsification method for federated LoRA fine-tuning based on importance scores. It effectively identifies and preserves the most salient parameters, overcoming the limitations of conventional sparsification methods.
- We propose a two-step server-side process: (1) reconstructing and aggregating client updates in a full-rank space to ensure mathematically sound aggregation and conflict mitigation, and (2) an alternating decomposition method to create sparse global updates for efficient download. We also introduce an efficient variant of this process to reduce server computation.
- We conduct extensive experiments demonstrating that our methods can save over 90% of communication costs while achieving superior model performance, validating FedSRD as a practical and high-performing solution for federated LLMs fine-tuning on the Web.

2 Related Works

2.1 Parameter-Efficient Fine-Tuning

The large scale of modern LLMs makes the full-parameter fine-tuning computationally prohibitive in resource-constrained environments. To address this, Parameter-Efficient Fine-Tuning (PEFT) methods [7] freeze most of model parameters and only update a small part. Current approaches include adapter-based methods [9, 12, 14, 15] that insert small, trainable modules in models, and prompting methods [21, 32], such as prefix-tuning [23], that add learnable tokens to the input sequence. Among these, Low-Rank Adaptation (LoRA) [15] has emerged as a dominant and effective approach. The key insight of LoRA is that weight updates in fine-tuning possess a low intrinsic rank. LoRA represents each weight matrix update as the product of two smaller, trainable low-rank matrices, significantly reducing the number of trainable parameters and memory footprint while achieving comparable performance.

2.2 Federated LLMs Fine-tuning

Federated Learning (FL) enables the collaborative fine-tuning of LLMs on decentralized private data, addressing both data scarcity and privacy concerns. A common approach is to combine PEFT methods with standard aggregation algorithms [19, 25, 29, 44, 46]. Recent studies point out that this naive combination introduces aggregation inaccuracy, where separately averaging the decomposed LoRA matrices (B and A) from clients is not mathematically equivalent to averaging the true underlying model updates (BA). Several advanced methods have been proposed to mitigate this issue. FFA-LoRA [37] ensures a mathematically sound aggregation by freezing the LoRA A matrices and only aggregating the B matrices. FLoRA [39] stacks clients' LoRA matrices to ensure accurate aggregation. FlexLoRA [3] proposes to aggregate the true model updates (BA) on the server and then employs Singular Value Decomposition (SVD) to derive new global low-rank matrices for clients. FedICU [24] disentangles client updates into consensus and divergence components, which are then aggregated separately.

2.3 Communication-Efficient Federated LLMs Finetuning

Communication overhead remains a primary bottleneck in FL systems [35, 41, 42], a problem that is exacerbated when fine-tuning LLMs over the public internet. While PEFT methods provide a baseline level of efficiency, further optimizations are crucial. Structured methods impose constraints on the LoRA updates. For instance, FFA-LoRA [37] halves the communication cost by transmitting only the B matrices. Fed-SB [36] freezes the LoRA matrices entirely and instead inserts a small, trainable square matrix for aggregation. While these approaches reduce communication, they often do so at the cost of model performance by constraining the expressive capacity of LoRA. Other lines of work focus on generic compression. Quantization methods [1, 18] represent parameters with fewer bits, but they often fail to achieve high compression ratios and can lead to significant accuracy degradation, particularly in non-IID settings. Sparsification methods, which transmit only a subset of non-zero parameters, can achieve much higher compression ratios [1, 33]. Yet, traditional approaches are not directly applicable to LoRA. Magnitude-based pruning [1] is suboptimal because it ignores the coupled relationship between the two LoRA matrices. Similarly, random sparsification [43, 45] fails to preserve critical parameters, resulting in inferior performance at high sparsity ratios.

3 Preliminaries

3.1 LoRA in LLMs Fine-tuning

LoRA has been proven to exhibit better performance than other PEFT methods in the federated learning system [19]. LoRA constrains the weight update in low-rank space:

$$W_0 + \Delta W = W_0 + BA, \tag{1}$$

where W_0 is the frozen model weights, $B \in \mathbb{R}^{d_{out} \times r}$ and $A \in \mathbb{R}^{r \times d_{in}}$ are paired low-rank trainable matrices inserted in the linear weights to approximate the update $\Delta W \in \mathbb{R}^{d_{out} \times d_{in}}$, where $r \ll \min(d_{out}, d_{in})$. B is initialized to zero and A is initialized with random Gaussian noises [15].

3.2 Problem Formulation

We consider an FL system with one server and m clients. Each client has its local private dataset $\mathcal{D}^i = \{x_j^i, y_j^i\}_{j=1}^{|\mathcal{D}^i|}$. These datasets may originate from different domains or tasks, creating a heterogeneous (non-IID) scenario. All clients collaboratively fine-tune a shared LLM $\mathcal{M}$ to perform well on their respective data distributions. This can be formulated as the following optimization problem:

$$\min_{\mathcal{M}} \mathcal{L}(\mathcal{M}) = \sum_{i}^{m} \mathcal{L}_i(\mathcal{M}, \mathcal{D}^i), \tag{2}$$

where $\mathcal{L}_i(\mathcal{M}, \mathcal{D}^i)$ represents the loss of the LLM $\mathcal{M}$ on local dataset $\mathcal{D}^i$. Following the recent studies, we adopt LoRA to fine-tune the LLM $\mathcal{M}$ in local fine-tuning on each client. The local optimization of client i could be formulated as:

$$\min \mathcal{L}_i = \min_{\mathcal{P}(\mathcal{M})} \mathbb{E}_{(x_j^i, y_j^i) \sim \mathcal{D}^i} L(x_j^i, y_j^i, \mathcal{P}(\mathcal{M})), \tag{3}$$

where $\mathcal{P}(\mathcal{M})$ denotes the trainable parameters set of LoRA. L is the cross-entroy loss on completion y_j^i given x_j^i.

4 Proposed Method

In this section, we introduce FedSRD, a noval Sparsify-Reconstruct-Decompose framework for federated LoRA fine-tuning, illustrated in Figure 3. At each communication round t, a client i first computes its local LoRA updates ΔB_i^t and ΔA_i^t, then uses our proposed importance-aware strategy to sparsify them for upload (Section 4.1). The server then reconstructs each client's full weight matrix and aggregates them in the full-rank space to mitigate non-IID conflicts (Section 4.2). Finally, to ensure an efficient download, the server decomposes the resulting global update into a single sparse low-rank matrix for broadcast using an alternating, Taylor approximation-based method (Section 4.3).

4.1 Importance-aware Sparsification

Motivation. To reduce upload communication costs, sparsification is a natural approach. However, mainstream methods are ill-suited for federated LoRA fine-tuning. Magnitude-based sparsification is ineffective as it prunes each LoRA matrix independently,

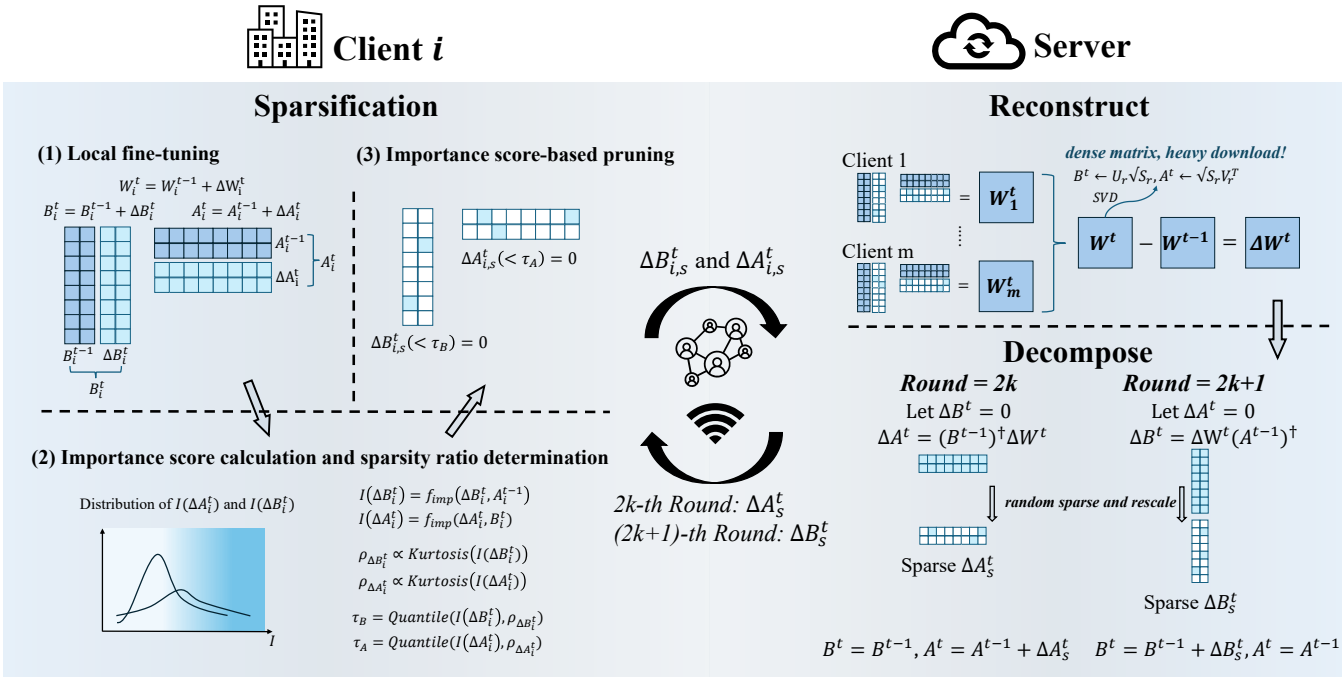

Figure 3: The FedSRD framework. (1) Client-Side sparsification: Each client computes its LoRA updates ΔB_i^t and ΔA_i^t and uses our importance-aware sparsification to generate sparse updates $\Delta B_{i,s}^t$ and $\Delta A_{i,s}^t$ for upload. (2) Server-Side reconstruction to aggregation: The server reconstructs each client's full weight matrix W_i^t and aggregates them in the full-rank space. (3) Server-Side decomposition: The server computes the global update ΔW^t, which is then decomposed into a single sparse matrix for efficient download.

ignoring the crucial structural interplay between B and A. On non-IID data, random-based sparsification fails to preserve performance at high sparsity ratios because it cannot differentiate between critical and unimportant parameters. This motivates a sparsification method that can (1) account for the structural relationship between B and A and (2) robustly preserve the most salient parameters.

To address these challenges, we propose importance-aware sparsification. Instead of transmitting the full LoRA matrices, we transmit only their updates at each round. Specifically, on client i at round t, we first conduct LoRA fine-tuning on its local dataset to obtain the new LoRA matrices B_i^t and A_i^t, where

$$B_i^t = B_i^{t-1} + \Delta B_i^t, \quad A_i^t = A_i^{t-1} + \Delta A_i^t, \tag{4}$$

where B_i^{t-1} and A_i^{t-1} are the matrices from the previous round [1]. The full-rank weight update ΔW_i^t can be expressed as:

$$\begin{aligned} \Delta W_i^t &= B_i^t A_i^t - B_i^{t-1} A_i^{t-1} \\ &= B_i^t (A_i^{t-1} + \Delta A_i^t) - B_i^{t-1} A_i^{t-1} \\ &= (B_i^t - B_i^{t-1}) A_i^{t-1} + B_i^t \Delta A_i^{t-1} \\ &= \Delta B_i^t A_i^{t-1} + B_i^t \Delta A_i^t. \end{aligned} \tag{5}$$

This formulation reveals that the true contribution of an element in ΔB_i^t depends on the corresponding row in A_i^{t-1}, and likewise for ΔA_i^t and B_i^t. To formalize this, we therefore define importance score $I(\cdot)$ to measure an element's contribution to the final weight update by scaling its magnitude with the norm of the corresponding vector it interacts with in the other matrix:

$$\begin{aligned} I(\Delta B_i^t[u,v]) &= |\Delta B_i^t[u,v]| \cdot \|A_i^{t-1}[v,:]^T\|_2, \\ I(\Delta A_i^t[u,v]) &= |\Delta A_i^t[u,v]| \cdot \|B_i^t[:,u]\|_2. \end{aligned} \tag{6}$$

A higher importance score indicates a greater contribution to the final weight update. We then prune elements with the lowest scores. We explain the rationale behind and prove that pruning based on this score minimizes the upper bound of the weight update approximation error $\|\Delta W_i^t\|_F$ in Appendix B.

Furthermore, recognizing that LoRA matrices in different layers play distinct roles [26], we adaptively assign a sparsity ratio ρ to each matrix. The core intuition is that the kurtosis of the importance score reveals a layer's functional "prunability". A high kurtosis (a leptokurtic, heavy-tailed distribution) signifies that importance is concentrated in a few parameters, justifying an aggressive sparsity ratio. Conversely, a low kurtosis (a platykurtic, flat distribution) implies that importance is diffused, necessitating a more conservative ratio. The layer-specific sparsity ratio is thus calculated as:

$$\begin{aligned} \rho_{\Delta B_i^t} &= \alpha + 0.1 \cdot \log(\text{kurtosis}(I(\Delta B_i^t))), \\ \rho_{\Delta A_i^t} &= \alpha + 0.1 \cdot \log(\text{kurtosis}(I(\Delta A_i^t))), \end{aligned} \tag{7}$$

where α is a hyperparameter. The sparsity ratios are truncated if they exceed a pre-set upper bound to ensure validity. We provide a

[1]Since our operations are layer independent, we omit layer-specific notations for clarity.

detailed explanation and justification of this design in Appendix E. We use the adaptive sparsity ratios to prune ΔB_i^t and ΔA_i^t:

$$(\Delta B_{i,s}^t)_{uv} = \begin{cases} (\Delta B_i^t)_{uv} & \text{if } I(\Delta B_i^t)_{uv} > \text{quantile}(I(\Delta B_i^t), \rho_{\Delta B_i^t}) \\ 0 & \text{otherwise} \end{cases}$$
$$(\Delta A_{i,s}^t)_{uv} = \begin{cases} (\Delta A_i^t)_{uv} & \text{if } I(\Delta A_i^t)_{uv} > \text{quantile}(I(\Delta A_i^t), \rho_{\Delta A_i^t}) \\ 0 & \text{otherwise} \end{cases}, \tag{8}$$

where the $\Delta B_{i,s}^t$ and $\Delta A_{i,s}^t$ are sparsified matrices for uploading. The client can upload the sparse matrices in a compressed format.

4.2 Reconstruct to Aggregation

Motivation. Most existing federated learning methods like FedAvg [27], when applied to LoRA, average the clients' LoRA matrices independently. This aggregation is not mathematically sound and introduces unexpected noises because its multiplicative relationship between the matrices and fails to mitigate update conflicts arising from non-IID client data [3, 37, 39]. This motivates us to develop an advanced aggregation method.

To enable a mathematically sound aggregation that preserves client knowledge, we propose to first reconstruct each client's update in the full-rank space. At round t, upon receiving the sparse updates $\Delta B_{i,s}^t$ and $\Delta A_{i,s}^t$ from m participating clients, the server first computes each client's new LoRA matrices, B_i^t and A_i^t, using Equation 4. It then reconstructs each client's full-rank weight matrix and aggregates them:

$$W^t = \frac{1}{m} \sum_i^m B_i^t A_i^t, \tag{9}$$

where W^t is the aggregated LoRA weights in the full-rank parameter space. By performing aggregation in this higher-dimensional space, the client-specific knowledge encoded in B_i^t and A_i^t is properly combined, reducing the likelihood of destructive interference.

4.3 Alternative Decomposition

Motivation. Previous works [3, 20] often use Singular Value Decomposition (SVD) to decompose aggregated weights into new dense matrices B^t and A^t. But transmitting them creates a download bottleneck. Naively sparsifying the full state matrices B^t and A^t would damage the learned model. Computing the update as a direct difference, $B^t - B^{t-1}$, is also mathematically flawed. This is because SVD is rotation-invariant and decomposition is not unique. Even a small global update can induce an arbitrary rotation of the low-rank basis between rounds, creating large, dense updates that do not reflect the true minimal updates. Thus, they are highly unsuitable for sparsification. These motivate the need for a novel decomposition capable of producing a stable update for sparse downloads.

Our goal is to find sparse updates ΔB^t and ΔA^t that approximate the change from the previous global state W^{t-1} to the new one W^t. First, we project W^t to its r-rank approximation via SVD:

$$\begin{aligned} U, S, V_h &= SVD(W^t), \\ W_{low_rank}^t &= U[:, :r] S[:r, :r] V_h[:r, :], \end{aligned} \tag{10}$$

where $W_{low_rank}^t$ is the r-rank approximation of W^t, r is the LoRA rank. This process inherently denoises the aggregated result by discarding less important components and retaining the consensus. Assuming the global states are low-rank, we can state that $\exists B^{t-1}, A^{t-1}$ such that $W_{\text{low_rank}}^{t-1} = B^{t-1}A^{t-1}$, and we seek $\Delta B^t, \Delta A^t$ such that $W_{\text{low_rank}}^t = (B^{t-1} + \Delta B^t)(A^{t-1} + \Delta A^t)$. We can approximate the update using a first-order Taylor expansion:

$$\begin{aligned} & W_{low_rank}^t - W_{low_rank}^{t-1} \\ &= (B^{t-1} + \Delta B^t)(A^{t-1} + \Delta A^t) - B^{t-1}A^{t-1} \\ &= B^{t-1}\Delta A^t + \Delta B^t A^{t-1} + \Delta B^t \Delta A^t \\ &\approx B^{t-1}\Delta A^t + \Delta B^t A^{t-1}, \end{aligned} \tag{11}$$

where the higher-order term $\Delta B^t \Delta A^t$ is omitted (discussed in Appendix F). To solve this underdetermined system, we alternate between setting one of the deltas to zero. In even-numbered rounds, we set $\Delta B^t = \mathbf{0}$ and solve for ΔA^t:

$$\begin{aligned} & W_{low_rank}^t - W_{low_rank}^{t-1} = B^{t-1}\Delta A^t, \\ & \Delta A^t = (B^{t-1})^\dagger (W_{low_rank}^t - W_{low_rank}^{t-1}), \end{aligned} \tag{12}$$

where $(B^{t-1})^\dagger$ is the Moore-Penrose pseudoinverse. In odd-numbered rounds, we set $\Delta A^t = \mathbf{0}$ and solve for ΔB^t:

$$\begin{aligned} & W_{low_rank}^t - W_{low_rank}^{t-1} = \Delta B^t A^{t-1}, \\ & \Delta B^t = (W_{low_rank}^t - W_{low_rank}^{t-1})(A^{t-1})^\dagger, \end{aligned} \tag{13}$$

The resulting single update matrix (ΔA^t or ΔB^t) can be simply sparsified using random pruning and rescaled [45] to produce a sparse update (ΔA_s^t or ΔB_s^t) for download. Clients receive this single sparse matrix and update their local model accordingly, completing the communication-efficient cycle.

FedSRD-e: a Computationally Efficient Variant. For scenarios where server-side computation is the primary bottleneck, such as in edge devices, we propose a computationally efficient variant. This variant, FedSRD-e, bypasses the SVD projection and instead directly approximates the full-rank global update, $\Delta W^t = W^t - W^{t-1}$. This matrix ΔW^t replaces $W_{\text{low_rank}}^t - W_{\text{low_rank}}^{t-1}$ on the left-hand side of Equation 11. The subsequent steps of solving for a single update matrix remain the same. This variant significantly reduces computation by avoiding the costly SVD operation. It only consumes slightly more time than the simple FedAvg method, serving as a powerful, high-speed alternative.

Remark. Note that both the reconstruction and decomposition operations are layer-independent, meaning they can be performed layer by layer. Thus, the additional memory resource demand on the server is negligible.

Table 1: Details of the local datasets used by each client. All datasets are from the training split.

Domain	Dataset	Task Type
Code	Code Alpaca	Code Generation
Medical	MedQA	Medical Question Answering
Finance	Finance Alpaca	Financial Question Answering
Math	GSM8K (train) Math (train)	Mathematical Reasoning & Math Problems Solving

Table 2: Details of the benchmarks used in our evaluation.

Category	Domain	Benchmark	# Instance	k-Shot
In-domain	Code	HumanEval [5]	164	0
		Sanitized MBPP [2]	257	3
	Medical	MedQA [16]	1,273	1
		MedMCQA [31]	4,183	1
	Finance	FinEval [47]	4,661 (34 sub-tasks)	0
		FinanceIQ [48]	7,173 (10 sub-tasks)	0
	Math	GSM8K [6]	1319	0
		MATH [13]	5000	0
Out-of-domain	General	AGIEval [49]	8,062 (20 sub-tasks)	0
	Law	LawBench [10]	10,000 (20 sub-tasks)	1

5 Experiments

5.1 Experimental Setup

5.1.1 Datasets. We simulate an FL system with 4 clients from distinct specialist domains: Code, Medical, Finance, and Math. We use CodeAlpaca [4], the training set of MedQA [16], Finance Alpaca[2], and a combination of the training sets from GMS8K [6] and MATH [13] as the local datasets for these clients, respectively.

5.1.2 Baselines. We compare our framework FedSRD and FedSRD-e with 6 state-of-the-art federated learning methods: (1) FedAvg [27], (2) FedProx [22], (3) FFA-LoRA [37], which freezes LoRA A matrcies to ensure efficient communication and effective aggregation, (4) FlexLoRA [3], which aggregates in full-rank space and uses SVD for download, (5) FedICU [24], which selectively updates on global model and perform consensus-divergence splitting in the server-side aggregatio, (6) Fed-DARE, which applies random sparsification with rescaling (DARE [45]) to LoRA updates for upload.

5.1.3 Benchmarks. We evaluate baselines and our methods on 10 benchmarks, including 8 in-domain benchmarks from Code, Medical, Finance, and Math domains, and 2 out-of-domain benchmarks to evaluate the knowledge retention of other domains during fine-tuning. We provide details of these benchmarks in Table 2.

5.1.4 Implementation. In our experiments, we adopt Llama3.2-3B [3] and Qwen2-7B [38] as base models for federated fine-tuning over 50 rounds. We apply LoRA to all linear layers with rank $r = 64$ for Llama3.2-3B and $r = 32$ for Qwen2-7B. The learning rate is tuned from 1e-5 to 2e-4. For Fed-DARE, the sparsity ratio is 0.9 as suggested [45]. For our method, we tune α in Equation 7 within [0.85, 0.9] and set the download sparsity ratio to 0.8. For other hyperparameters, we follow the suggested values from the original papers. We report average performance on in-domain and out-of-domain benchmarks, as well as the per-round communication cost per client. We conduct training on NVIDIA A800 GPUs and inference on NVIDIA GeForce RTX 3090 GPUs with bfloat16 precision.

5.2 Experimental Results

5.2.1 Performance Comparison. The main results are reported in Table 3. We analyze the results from three key perspectives.

[2]https://huggingface.co/datasets/gbharti/finance-alpaca
[3]https://huggingface.co/meta-llama/Llama-3.2-3B

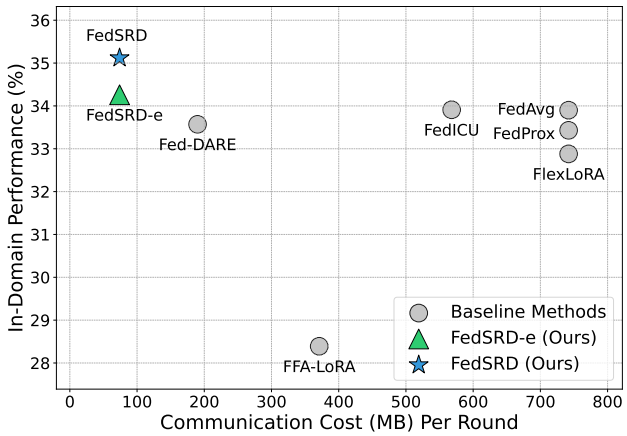

Figure 4: Average in-domain performance vs. per-round communication cost (Llama3.2-3B). Our methods are Pareto optimal, achieving the highest performance with the lowest communication cost.

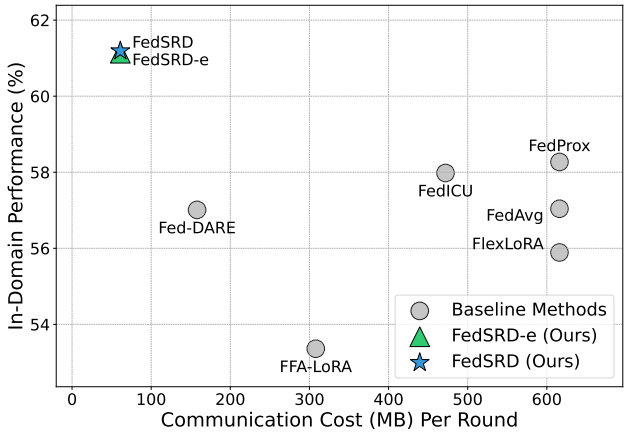

Figure 5: Average in-domain performance vs. per-round communication cost (Qwen2-7B). Our methods are Pareto optimal, achieving the highest performance with the lowest communication cost.

In-Domain Performance. Across the experiments, our proposed methods, FedSRD and FedSRD-e, consistently achieve state-of-the-art performance on in-domain benchmarks. As shown in Table 3, with the Llama3.2-3B model, FedSRD and FedSRD-e achieve the highest average in-domain performance score, outperforming all baseline methods. This trend is further confirmed with the larger Qwen2-7B model, where FedSRD and FedSRD-e again lead with top performance, respectively. While some baselines may achieve peak performance on specific benchmarks, they often falter in other domains. In contrast, our framework exhibits robust and consistently high performance across all four specialized domains, underscoring its superior capability in aggregating diverse knowledge from specialist clients to build powerful global LLMs.

Table 3: Comparison of model performance and per-round communication cost. The top section uses Llama3.2-3B and the bottom section uses Qwen2-7B. OOD represents Out-of-Domain. Best and second-best results are bolded and underlined.

Method	Comm. Cost (MB)	Average		In-Domain								Out-of-Domain	
				Code		Medical		Finance		Math			
		In-Domain	OOD	HumanEval	MBPP	MedQA	MedMCQA	FinEval	FinanceIQ	GSM8K	MATH	AGIEval	Law
Base Model: Llama3.2-3B													
FedAvg	742	33.90	24.36	26.85	47.86	40.45	41.53	41.96	38.08	28.96	5.56	22.47	26.24
FedProx	742	33.43	23.82	28.05	48.25	35.98	40.06	42.75	37.82	28.05	5.70	21.58	26.06
FFA-LoRA	371	28.39	26.12	3.05	46.30	39.95	40.11	40.23	36.02	24.79	5.10	24.63	**27.60**
FlexLoRA	742	32.36	26.08	28.75	47.47	36.29	41.24	42.22	37.09	27.37	5.68	24.92	27.24
FedICU	568	33.91	25.76	**31.10**	**49.42**	40.15	41.07	40.83	37.56	25.09	5.26	24.77	26.75
Fed-DARE	190	33.57	24.25	26.60	48.25	39.28	40.04	40.40	38.04	26.05	5.26	21.94	26.56
FedSRD-e	**74**	34.26	**27.95**	29.27	45.87	41.63	41.17	42.22	**39.47**	**30.48**	**5.88**	**28.81**	27.08
FedSRD	**74**	**35.12**	26.17	**31.10**	**49.42**	**41.95**	**41.62**	**43.44**	39.13	28.96	5.36	25.54	26.79
Base Model: Qwen2-7B													
FedAvg	616	57.04	41.02	59.15	37.35	61.67	75.23	65.16	49.04	64.29	44.46	48.25	33.79
FedProx	616	58.27	40.72	63.42	36.58	62.13	77.34	65.77	49.96	66.26	44.72	**48.62**	32.82
FFA-LoRA	308	53.36	**43.41**	**68.29**	17.90	51.55	57.02	**67.76**	48.92	69.37	**46.08**	46.92	**39.89**
FlexLoRA	616	55.89	40.49	63.41	35.02	58.83	68.30	67.68	50.53	58.61	44.70	47.81	33.17
FedICU	472	57.98	42.56	60.37	**38.91**	60.33	78.72	65.86	48.83	66.19	44.62	48.44	36.28
Fed-DARE	158	57.01	41.72	60.37	31.91	61.12	75.30	66.03	**50.84**	65.96	44.58	48.37	35.07
FedSRD-e	**61**	61.13	42.59	68.09	36.56	**77.80**	78.10	64.21	48.90	**70.20**	45.20	45.82	39.35
FedSRD	**61**	**61.19**	42.58	**68.29**	36.19	77.77	**78.94**	64.55	48.48	69.93	45.36	45.93	39.22

Communication Efficiency. A standout advantage of our framework is its remarkable communication efficiency. FedSRD and FedSRD-e reduce the per-round communication overhead to just 74 MB for Llama3.2-3B and 61 MB for Qwen2-7B, an approximate 90% reduction compared to standard methods like FedAvg (742 MB) and advanced methods like FlexLoRA (742 MB). This is achieved by our proposed *importance-aware sparsification* for uploads and *alternative decomposition* for downloads (see Appendix C for a detailed analysis). Crucially, this significant gain in efficiency does not compromise performance. On the contrary, FedSRD and FedSRD-e achieve higher performance than baselines, directly answering our central research question. This is visualized in Figures 4 and 5, where our methods are shown to be Pareto optimal: achieving the best performance with the least communication.

Out-of-Domain Performance. Our methods also show strong performance on out-of-domain (OOD) benchmarks, effectively mitigating catastrophic forgetting. FedSRD and FedSRD-e achieve leading results on Llama3.2-3B and remain competitive on Qwen2-7B. Although FFA-LoRA exhibits strong OOD retention, we attribute this to its mechanism of freezing the LoRA A matrices. which inherently constrains the capacity and hinders new knowledge acquisition in favor of preserving prior information. In contrast, our methods provide a more balanced and consistently high performance across both in-domain and out-of-domain evaluations. Further, we posit that the SVD projection in FedSRD, by seeking a low-rank consensus for in-domain tasks, may inadvertently filter out some general knowledge from the base model, which could account for the superior OOD performance of FedSRD-e. In summary, our framework not only excels at integrating specialized knowledge but also effectively preserves the general capabilities of the base model.

5.2.2 Abation Study. To validate each component of our FedSRD framework, we conducted an ablation study on Llama3.2-3B, with results in Table 4. Starting with the full model, we sequentially removed or replaced key components. Disabling the *adaptive sparsity ratio* (using a fixed ratio instead) causes a slight performance drop, affirming the benefit of dynamic adjustment. Replacing our *importance-aware sparsification* with a random baseline [45] leads to a significant degradation, highlighting that our decomposition method effectively preserves aggregated knowledge while reducing download communication costs. Further, ablating our *alternative decomposition* (reverting to dense SVD download) also harms performance, confirming its effectiveness in preserving knowledge while reducing download costs. Finally, removing the reconstruction step (reverting to naive LoRA matrix averaging) results in severe performance loss, proving that full-rank aggregation is crucial for mitigating client heterogeneity. These results confirm that each component plays an indispensable and synergistic role in achieving the dual objectives of high model performance and extreme communication efficiency. Additionally, we show in Appendix D that our importance-aware sparsification also works as a more effective plugin to enhance conventional methods like FedAvg.

5.2.3 Impact of Sparsification Ratios. We first analyze the uplink base sparsity ratio, α, in our importance-aware sparsification. As illustrated in Figure 6, FedSRD is robust and consistently outperforms all baselines even at an extreme sparsity of 98% (α=0.95). Interestingly, OOD performance improves with a higher α. This suggests that uploading and aggregating fewer parameters helps preserve the pretrained model's generalizable knowledge by filtering out potentially conflicts in client-specific updates, thus preventing catastrophic forgetting. We can also observe that smaller α values

Table 4: Ablation study on Llama3.2-3B. Components are removed sequentially. Note that *importance-aware sparsification* is substituted with the random sparsification baseline [45] for its ablation.

Method	In-Domain	OOD
FedSRD	35.12	26.17
– *adaptive sparsity ratio* (Eq. 7)	35.05	25.94
– *importance-aware sparsification* (Sec. 4.1)	32.93	24.76
– *alternative decomposition* (Sec. 4.3)	32.48	25.73
– *reconstruct* (Sec. 4.2)	32.32	23.90

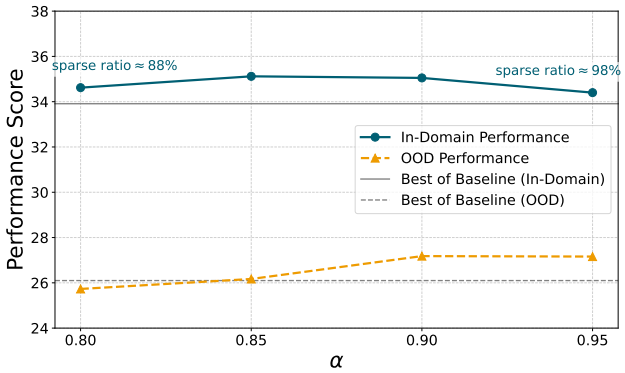

Figure 6: In-domain and out-of-domain (OOD) performance under different base sparsity values (α) for FedSRD. Performance is robust across a range of sparsity levels, consistently outperforming the best baseline results.

Table 5: In-domain and out-of-domain (OOD) performance under different download sparsification for FedSRD. Param. Reduction denotes the reduction ratio of the number of parameters downloaded per round.

Download Sparse Ratio	In-Domain	OOD	Param. Reduction
60%	34.32	26.76	80%
80%	35.12	26.17	90%
90%	34.57	25.13	95%

(corresponding to lower sparsity) do not necessitate better performance, since preserving more parameters may introduce irrelevant or noisy knowledge for aggregation. A properly set α therefore offers a dual advantage: it significantly reduces communication costs while simultaneously reducing the potential interference for more effective aggregation.

For the downlink, we apply random sparsification to the decomposed matrices. Table 5 shows that our method could even achieve better performance under aggressive sparsification. These results confirm that FedSRD achieves highly efficient communication and improved performance for federated LLMs fine-tuning.

Table 6: Computation time per round for different methods.

Time per Round (min)	Llama3.2-3B	Qwen2-7B
FedAvg	1.92	3.06
FedProx	2.10	3.18
FFA-LoRA	1.86	3.00
FlexLoRA	7.20	12.60
FedICU	30.36	51.60
Fed-DARE	1.92	3.06
FedSRD	7.92	13.50
FedSRD-e	2.10	3.15

5.3 Computation Time Analysis

We report the wall-clock time required for a single FL round in our simulation environment for both the Llama3.2-3B and Qwen2-7B in Table 6. Among the baselines, FedAvg provides a baseline for time consumption. FedProx requires slightly more time due to its proximal term, whereas FFA-LoRA is faster as it only trains and aggregates half of the LoRA matrices. FlexLoRA consumes more time performing SVD for matrix decomposition, and FedICU is the most time-intensive due to its computationally inefficient disentangled aggregation. For our methods, while the full FedSRD requires more time for its r-rank SVD projection, our efficient variant, **FedSRD-e, introduces negligible additional computation compared to FedAvg, while exhibiting performance second only to the full FedSRD**. This result highlights that our FedSRD-e variant achieves a Pareto-optimal balance across model performance, communication efficiency, and time consumption, making it a highly practical and scalable solution for real-world federated LLMs fine-tuning.

6 Conclusion

In this paper, we address the critical challenge of communication overhead in federated LoRA fine-tuning for LLMs. We propose FedSRD, a novel Sparsify-Reconstruct-Decompose framework for communication-efficient federated LLMs fine-tuning. Its core components include a structure-aware, importance-based sparsification strategy on the client side; a conflict-mitigating reconstruction and aggregation step in the full-rank space on the server side; and a novel alternating decomposition method to produce sparse global updates for efficient download. We also introduce FedSRD-e, a computationally light variant for resource-constrained environments. Through extensive experiments on diverse benchmarks, we demonstrated that FedSRD reduces communication costs by over 90% while simultaneously improving model performance compared to state-of-the-art methods. Our work presents a practical and high-performing solution that significantly lowers the barrier for scalable, collaborative AI development on the decentralized Web.

Acknowledgments

This work was supported by the National Key R&D Program of China under Grant No. 2022YFB2703301. This work was also supported by the Science and Technology Development Program of Two Districts in Xinjiang, China under Grant No. 2024LQ03004.

References

[1] Alham Fikri Aji and Kenneth Heafield. 2017. Sparse communication for distributed gradient descent. *arXiv preprint arXiv:1704.05021* (2017).

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Jiamu Bai, Daoyuan Chen, Bingchen Qian, Liuyi Yao, and Yaliang Li. 2024. Federated Fine-tuning of Large Language Models under Heterogeneous Tasks and Client Resources. *arXiv preprint arXiv:2402.11505* (2024).

[4] Sahil Chaudhary. 2023. Code Alpaca: An Instruction-following LLaMA model for code generation. https://github.com/sahil280114/codealpaca.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).

[7] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence* 5, 3 (2023), 220–235.

[8] Ron Dorfman, Shay Vargaftik, Yaniv Ben-Itzhak, and Kfir Yehuda Levy. 2023. DoCoFL: Downlink compression for cross-device federated learning. In *International Conference on Machine Learning*. PMLR, 8356–8388.

[9] Ali Edalati, Marzieh Tahaei, Ivan Kobyzev, Vahid Partovi Nia, James J Clark, and Mehdi Rezagholizadeh. 2025. KronA: Parameter-Efficient Tuning with Kronecker Adapter. In *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques*. Springer, 49–65.

[10] Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Songyang Zhang, Kai Chen, Zongwen Shen, and Jidong Ge. 2023. Lawbench: Benchmarking legal knowledge of large language models. *arXiv preprint arXiv:2309.16289* (2023).

[11] Solomon Golomb. 1966. Run-length encodings (corresp.). *IEEE transactions on information theory* 12, 3 (1966), 399–401.

[12] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2021. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366* (2021).

[13] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874* (2021).

[14] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*. PMLR, 2790–2799.

[15] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[16] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences* 11, 14 (2021), 6421.

[17] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[18] Jakub Konečnỳ, H Brendan McMahan, Felix X Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. 2016. Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492* (2016).

[19] Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5260–5271.

[20] Seanie Lee, Sangwoo Park, Dong Bok Lee, Dominik Wagner, Haebin Seong, Tobias Bocklet, Juho Lee, and Sung Ju Hwang. 2025. FedSVD: Adaptive Orthogonalization for Private Federated Learning with LoRA. *arXiv preprint arXiv:2505.12805* (2025).

[21] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).

[22] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems* 2 (2020), 429–450.

[23] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).

[24] Yangxu Liao, Wenke Huang, Guancheng Wan, Jian Liang, Bin Yang, and Mang Ye. [n. d.]. Splitting with Importance-aware Updating for Heterogeneous Federated Learning with Large Language Models. In *Forty-second International Conference on Machine Learning*.

[25] Guodong Long, Tao Shen, Jing Jiang, Michael Blumenstein, et al. 2024. Dual-personalizing adapter for federated foundation models. *Advances in Neural Information Processing Systems* 37 (2024), 39409–39433.

[26] Yingfeng Luo, Dingyang Lin, Junxin Wang, Ziqiang Xu, Kaiyan Chang, Tong Zheng, Bei Li, Anxiang Ma, Tong Xiao, Zhengtao Yu, et al. 2025. One Size Does Not Fit All: A Distribution-Aware Sparsification for More Precise Model Merging. *arXiv preprint arXiv:2508.06163* (2025).

[27] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.

[28] Thomas Mundt. 2004. How much is a byte?: a survey of costs for mobile data transmission. In *ACM International Conference Proceeding Series*, Vol. 58. 1–6.

[29] Duy Phuong Nguyen, J Pablo Munoz, and Ali Jannesari. 2024. Flora: Enhancing vision-language models with parameter-efficient federated learning. *arXiv preprint arXiv:2404.15182* (2024).

[30] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[31] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on health, inference, and learning*. PMLR, 248–260.

[32] Aleksandar Petrov, Philip HS Torr, and Adel Bibi. 2023. When do prompting and prefix-tuning work? a theory of capabilities and limitations. *arXiv preprint arXiv:2310.19698* (2023).

[33] Atal Sahu, Aritra Dutta, Ahmed M Abdelmoniem, Trambak Banerjee, Marco Canini, and Panos Kalnis. 2021. Rethinking gradient sparsification as total error minimization. *Advances in Neural Information Processing Systems* 34 (2021), 8133–8146.

[34] Lorenzo Sani, Alex Iacob, Zeyu Cao, Bill Marino, Yan Gao, Tomas Paulik, Wanru Zhao, William F Shen, Preslav Aleksandrov, Xinchi Qiu, et al. [n. d.]. The Future of Large Language Model Pre-training is Federated. In *International Workshop on Federated Foundation Models in Conjunction with NeurIPS 2024*.

[35] Felix Sattler, Simon Wiedemann, Klaus-Robert Müller, and Wojciech Samek. 2019. Robust and communication-efficient federated learning from non-iid data. *IEEE transactions on neural networks and learning systems* 31, 9 (2019), 3400–3413.

[36] Raghav Singhal, Kaustubh Ponkshe, Rohit Vartak, Lav R Varshney, and Praneeth Vepakomma. 2025. Fed-SB: A silver bullet for extreme communication efficiency and performance in (private) federated lora fine-tuning. *arXiv preprint arXiv:2502.15436* (2025).

[37] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. 2024. Improving lora in privacy-preserving federated learning. *arXiv preprint arXiv:2403.12313* (2024).

[38] Qwen Team. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* 2 (2024).

[39] Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li. 2024. Flora: Federated fine-tuning large language models with heterogeneous low-rank adaptations. *Advances in Neural Information Processing Systems* 37 (2024), 22513–22533.

[40] Yebo Wu, Chunlin Tian, Jingguang Li, He Sun, Kahou Tam, Zhanting Zhou, Haicheng Liao, Zhijiang Guo, Li Li, and Chengzhong Xu. 2025. A survey on federated fine-tuning of large language models. *arXiv preprint arXiv:2503.12016* (2025).

[41] Guochen Yan, Xunkai Li, Luyuan Xie, Wentao Zhang, Qingni Shen, Yuejian Fang, and Zhonghai Wu. 2024. Towards federated graph learning in one-shot communication. *arXiv preprint arXiv:2411.11304* (2024).

[42] Guochen Yan, Luyuan Xie, Xinyi Gao, Wentao Zhang, Qingni Shen, Yuejian Fang, and Zhonghai Wu. 2025. Fedvck: Non-iid robust and communication-efficient federated learning via valuable condensed knowledge for medical image analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 21904–21912.

[43] Yan Yang, Yixia Li, Hongru Wang, Xuetao Wei, Jianqiao Yu, Yun Chen, and Guanhua Chen. 2025. Impart: Importance-aware delta-sparsification for improved model compression and merging in llms. *arXiv preprint arXiv:2504.13237* (2025).

[44] Rui Ye, Wenhao Wang, Jingyi Chai, Dihan Li, Zexi Li, Yinda Xu, Yaxin Du, Yanfeng Wang, and Siheng Chen. 2024. Openfedllm: Training large language models on decentralized private data via federated learning. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 6137–6147.

[45] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2024. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*.

[46] Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. 2024. Towards building the federatedgpt: Federated instruction tuning. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 6915–6919.

[47] Liwen Zhang, Weige Cai, Zhaowei Liu, Zhi Yang, Wei Dai, Yujie Liao, Qianru Qin, Yifei Li, Xingyu Liu, Zhiqiang Liu, et al. 2023. Fineval: A chinese financial domain knowledge evaluation benchmark for large language models. *arXiv preprint arXiv:2308.09975* (2023).

[48] Xuanyu Zhang and Qing Yang. 2023. Xuanyuan 2.0: A large chinese financial chat model with hundreds of billions parameters. In *Proceedings of the 32nd ACM international conference on information and knowledge management*. 4435–4439.

[49] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. 2023. Agieval: A human-centric benchmark for evaluating foundation models. *arXiv preprint arXiv:2304.06364* (2023).

A Pseudo-Code of FedSRD and FedSRD-e

Algorithm 1 FedSRD and FedSRD-e Framework

Server Initializes: Global LoRA matrices B^0, A^0. Let C be the set of all clients.
for each communication round $t = 1, 2, \ldots, T$ **do**
  if $t > 1$ **then**
    Server broadcasts ΔB_s^{t-1} or ΔA_s^{t-1}.
    for each client $i \in C$ **in parallel do**
      Client updates local model: $B_i^{t-1} \leftarrow B_i^{t-2} + \Delta B_s^{t-1}$, $A_i^{t-1} \leftarrow A_i^{t-2} + \Delta A_s^{t-1}$.
    end for
  end if
  for each client $i \in C$ **in parallel do**
    $\Delta B_{i,s}^t, \Delta A_{i,s}^t \leftarrow \text{ClientTraining}(i)$
  end for
  Server-Side Reconstruction and Aggregation:
  for each client $i \in C$ **do**
    Client reconstructs its new state B_i^t, A_i^t using Eq. 4
    Server computes client's full-rank matrix: $W_i^t \leftarrow B_i^t A_i^t$.
  end for
  Aggregate in full-rank space: $W^t \leftarrow \frac{1}{|C|} \sum_i W_i^t$.
  Server-Side Alternating Decomposition:
  $W_r^t \leftarrow \text{SVD}_r(W^t)$. {FedSRD-e bypasses this SVD step.}
  Let $\Delta W_{update}^t = W_r^t - B^{t-1}A^{t-1}$. {For FedSRD-e, use W^t instead of W_r^t.}
  if t is even **then**
    $\Delta A^t \leftarrow (B^{t-1})^\dagger (\Delta W_{update}^t)$
    $\Delta B^t \leftarrow \mathbf{0}$
  else
    $\Delta B^t \leftarrow (\Delta W_{update}^t)(A^{t-1})^\dagger$
    $\Delta A^t \leftarrow \mathbf{0}$
  end if
  Update global state: $B^t \leftarrow B^{t-1} + \Delta B^t$, $A^t \leftarrow A^{t-1} + \Delta A^t$.
  Sparsify for download: $\Delta B_s^t, \Delta A_s^t \leftarrow \text{DARE}(\Delta B^t, \Delta A^t)$.
end for

Function ClientTraining(i):
  Fine-tune model on local data $\mathcal{D}_i$ to get new B_i', A_i'.
  Compute local updates:$\Delta B_i^t \leftarrow B_i' - B_i^{t-1}$, $\Delta A_i^t \leftarrow A_i' - A_i^{t-1}$.
  Importance-Aware Sparsification:
  Calculate importance scores $I(\Delta B_i^t), I(\Delta A_i^t)$ using Eq. 6.
  Calculate adaptive sparsity ratios $\rho_{\Delta B_i^t}, \rho_{\Delta A_i^t}$.
  Prune updates to get sparse $\Delta B_{i,s}^t, \Delta A_{i,s}^t$ adaptively.
return $\Delta B_{i,s}^t, \Delta A_{i,s}^t$ to server.

B Rationale of the Importance Score Calculation

We provide the mathematical motivation for the importance score defined in Equation 6. The core idea is to quantify how much a single parameter's change in a LoRA update matrix (ΔB_i^t or ΔA_i^t) contributes to the overall change in the full-rank weight matrix (ΔW_i^t).

Let's first recall the expression of the weight update:

$$\Delta W_i^t = \Delta B_i^t A_i^{t-1} + B_i^t \Delta A_i^t \tag{14}$$

We will analyze the contribution of a single element in ΔB_i^t to the term $\Delta B_i^t A_i^{t-1}$. The analysis for ΔA_i^t is symmetric.

A fundamental property of matrix multiplication is that the product of two matrices, $W = BA$, can be expressed as the sum of the outer products of the columns of B and the rows of A. If $B \in \mathbb{R}^{d\times r}$ has columns $\mathbf{b}_j$ and $A \in \mathbb{R}^{r\times k}$ has rows $\mathbf{a}_j^T$, then:

$$W = BA = \sum_{j=1}^{r} \mathbf{b}_j \otimes \mathbf{a}_j^T \tag{15}$$

This expansion reveals the structural interplay between the two matrices: the final matrix W is constructed from pairs of columns from B and rows from A.

Now, let's consider the contribution of a single scalar element, $(\Delta B_i^t)_{uv}$ (the element in row u and column v), to the matrix product $\Delta B_i^t A_i^{t-1}$. We can isolate this contribution by considering a change in only this single element. The effect of this single element on the product can be represented as:

$$(\Delta B_i^t)_{uv} \cdot (\mathbf{e}_u \otimes \mathbf{e}_v^T) \cdot A_i^{t-1} \tag{16}$$

where $\mathbf{e}_u$ and $\mathbf{e}_v$ are standard basis vectors (vectors of zeros with a single 1 at position u or v, respectively). Using the associativity of matrix multiplication, this becomes:

$$(\Delta B_i^t)_{uv} \cdot \mathbf{e}_u \otimes (\mathbf{e}_v^T A_i^{t-1}) \tag{17}$$

The term $\mathbf{e}_v^T A_i^{t-1}$ is a vector product that selects the v-th row of the matrix A_i^{t-1}, which we denote as $(A_i^{t-1})_{v,:}$. Thus, the contribution of the single element $(\Delta B_i^t)_{uv}$ to the final update matrix is itself a rank-1 matrix:

$$\text{Contribution Matrix} = (\Delta B_i^t)_{uv} \cdot (\mathbf{e}_u \otimes (A_i^{t-1})_{v,:}) \tag{18}$$

This matrix is zero everywhere except for its u-th row, which is the v-th row of A_i^{t-1} scaled by the scalar $(\Delta B_i^t)_{uv}$.

A natural way to measure the magnitude, or "importance", of this contribution matrix is to compute its Frobenius norm ($\|\cdot\|_F$). The Frobenius norm of a scaled outer product of two vectors $\mathbf{x}$ and $\mathbf{y}$ is $|c| \cdot \|\mathbf{x}\|_2 \cdot \|\mathbf{y}\|_2$. In our case, $c = (\Delta B_i^t)_{uv}$, $\mathbf{x} = \mathbf{e}_u$, and $\mathbf{y} = (A_i^{t-1})_{v,:}$. Since $\|\mathbf{e}_u\|_2 = 1$, the norm is:

$$\begin{aligned} I((\Delta B_i^t)_{uv}) &= \left\|(\Delta B_i^t)_{uv} \cdot (\mathbf{e}_u \otimes (A_i^{t-1})_{v,:})\right\|_F \\ &= |(\Delta B_i^t)_{uv}| \cdot \|\mathbf{e}_u\|_2 \cdot \|(A_i^{t-1})_{v,:}\|_2 \\ &= |(\Delta B_i^t)_{uv}| \cdot \|(A_i^{t-1})_{v,:}^T\|_2 \end{aligned} \tag{19}$$

This is precisely the importance score for an element in ΔB_i^t as defined in Equation 6. A symmetric derivation for the term $B_i^t \Delta A_i^t$ shows that the importance of $(\Delta A_i^t)_{uv}$ is its magnitude multiplied by the L2-norm of the u-th column of B_i^t. This formulation provides

Table 7: Average In-domain Performance Comparison

	Performance
FedAvg	33.90
with Random sparsification (i.e., Fed-DARE)	33.57
with Magnitude sparsification	31.61
with **Importance-aware sparsification (Our proposed)**	**34.04**

Table 8: Communication cost comparison per round (MB). Costs include both parameters and bitmaps where applicable.

Method	Upload (MB)	Download (MB)	Total (MB)
FedAvg / FedProx / FlexLoRA	371	371	742
FFA-LoRA	185.5	185.5	371
FedICU	≈197	371	≈568
Fed-DARE	≈48.7	≈141.5	≈190
Ours (FedSRD / FedSRD-e)	**≈31.0**	**≈43.0**	**≈74.0**

a principled way to measure a parameter's true structural contribution to the final update, moving beyond its simple magnitude.

B.1 Theoretical Analysis of Pruning Error

In this section, we provide a theoretical justification for the proposed importance score. We demonstrate that our greedy pruning strategy, based on the importance score defined in Equation 6, is mathematically equivalent to minimizing the upper bound of the reconstruction error of the full-rank weight update ΔW.

B.1.1 Error Bound Derivation. We analyze the approximation error for the term $\Delta W_B = \Delta B_i^t A_i^{t-1}$ (the derivation for ΔA is symmetric). Let $\mathcal{S}$ denote the set of indices in ΔB_i^t selected for pruning. The pruning error matrix $\mathcal{E}$ is the sum of the rank-1 contributions of the removed elements:

$$\mathcal{E} = \sum_{(u,v)\in\mathcal{S}} (\Delta B_i^t)_{uv} \cdot (\mathbf{e}_u \otimes \mathbf{a}_v^T), \tag{20}$$

where $\mathbf{a}_v^T$ is the v-th row of A_i^{t-1}. By applying the Triangle Inequality to the Frobenius norm of $\mathcal{E}$, we derive the error upper bound:

$$\begin{aligned} \|\mathcal{E}\|_F &= \left\| \sum_{(u,v)\in\mathcal{S}} (\Delta B_i^t)_{uv} (\mathbf{e}_u \otimes \mathbf{a}_v^T) \right\|_F \\ &\leq \sum_{(u,v)\in\mathcal{S}} \underbrace{|(\Delta B_i^t)_{uv}| \cdot \|\mathbf{a}_v\|_2}_{I(\Delta B_i^t[u,v])}. \end{aligned} \tag{21}$$

The right-hand side is exactly the sum of the importance scores of the pruned parameters. Thus, pruning elements with the lowest scores $I(\cdot)$ strictly minimizes the upper bound of the weight update reconstruction error $\|\Delta W\|_F$.

B.1.2 Comparison with Gradient-based Pruning. Gradient-based pruning methods are impractical for Federated LoRA Fine-tuning for several reasons:

- **Computational and Memory Overhead:** Calculating the exact influence of parameters on the loss usually requires second-order derivatives (e.g., Hessian). For LLMs, computing or even approximating the Hessian is prohibitively expensive for resource-constrained edge devices. Even first-order methods require retaining gradients, doubling the memory footprint during the scoring phase.
- **Federated Constraints:** In a federated setting, data is non-IID. A gradient-based score computed on local data may not accurately reflect the parameter's importance to the global model, potentially leading to overfitting local distributions.

In contrast, our proposed importance score is a *data-free* and *zero-overhead* metric. It relies solely on the magnitude of the weights and the structural correlations within the LoRA matrices (ΔB and A), which are already present in memory. This ensures that our method minimizes the deviation in the weight space ($\|\Delta W\|$) efficiently, offering a superior trade-off between communication efficiency and model performance.

C Communication Cost Analysis

We analyze the communication cost per round for each method, using a Llama3.2-3B model fine-tuned with LoRA (rank $r = 64$) as a case study. The model contains 97,255,424 LoRA parameters stored in float32 format. The baseline payload for transmitting the full, dense parameter set in one direction (upload or download) is $\frac{97{,}255{,}424 \times 4 \text{ bytes}}{1024^2 \text{ bytes/MB}} = 371$ MB. For sparse methods, an additional bitmap is required to encode the positions of non-zero parameters, costing $\frac{97{,}255{,}424 \text{ bits}}{8 \text{ bits/byte} \times 1024^2 \text{ bytes/MB}} \approx 11.6$ MB for the full set.

Table 8 details the communication costs for each method. Baseline approaches like FedAvg, FedProx, and FlexLoRA require a full round-trip transmission, totaling 742 MB. FFA-LoRA halves this cost by communicating only the LoRA B matrices. FedICU uses a selective upload (≈50% sparsity) and a dense download, for a total of 568 MB. Fed-DARE, with a 90% sparsity on uploads and an aggregated slightly more than 60% sparsity on downloads, lowers the cost to 190 MB. Our proposed method achieves the highest communication efficiency. Client-side importance-aware pruning results in a lean 31 MB upload. For download, the server transmits only half of the parameters (alternating between LoRA A and B matrices) with 80% sparsity, leading to a download cost of just 43 MB. This culminates in a total round-trip cost of only 74 MB, representing a 90% reduction compared to FedAvg. The overhead from bitmaps can be further mitigated using lossless compression techniques like Run-Length Encoding [11].

D Importance-aware Sparsification Suits Well for FedAvg

To validate the superiority of our proposed importance-aware sparsification over others, we plug and compare them on FedAvg in Table 7. The results show that our importance-aware sparsification is more effective in the FL system.

E Justification for Equation 7

Equation 7 proposes a data-driven approach to dynamically determine the layer-wise sparsity ratio, ρ, for the LoRA update matrices ΔA_i^t and ΔB_i^t. The formulation is designed to adapt the pruning aggressiveness based on the intrinsic characteristics of each layer's importance score distribution, rather than relying on a uniform, manually-tuned sparsity level. The rationale behind each component of the equation is as follows:

Table 9: Performance of base models and local training

	Llama3.2-3B		Qwen2-7B	
Model	In-Domain	OOD	In-Domain	OOD
Base	16.33	14.90	44.79	38.83
Code	29.74	24.19	48.93	41.84
Medical	25.49	23.65	52.11	40.92
Finance	27.73	23.33	49.44	35.29
Math	19.25	18.80	43.83	34.16

Kurtosis as a Heuristic for Prunability: The core of our adaptive mechanism lies in using the kurtosis of the importance score distribution, $I(\cdot)$, as a quantitative indicator of a layer's "prunability." Kurtosis measures the "tailedness" of a distribution. A high kurtosis (a leptokurtic distribution) signifies that the importance scores are heavily concentrated on a small subset of parameters. This indicates a high degree of functional specialization and redundancy within the layer, making it an ideal candidate for aggressive pruning with minimal impact on performance. Conversely, a low kurtosis (a platykurtic distribution) implies that importance is more evenly diffused across many parameters. In this scenario, aggressive pruning would be detrimental, as it would likely remove parameters that make a non-trivial contribution. Therefore, a more conservative sparsity ratio is needed. By linking the sparsity ratio directly to kurtosis, our method automatically allocates a higher pruning budget to layers that can tolerate it and protects those that cannot.

Logarithmic Scaling: The raw values of kurtosis can span a wide range across different layers and training steps. We employ a logarithmic function to compress the wide range of kurtosis values into a more stable and manageable scale. This ensures that while a higher kurtosis leads to a higher sparsity ratio, the relationship is less sensitive to outlier kurtosis values, promoting more stable and robust pruning behavior throughout the process.

Modulation Coefficient: This coefficient scales the log-kurtosis term, modulating the final sparsity ratio. Empirically, we observe kurtosis values mostly ranging from 1 to 12 (with extremes up to 40). We selected a coefficient of 0.1. This choice is well-justified as it maps the typical kurtosis range to a moderate adjustment interval of approximately [0, 0.25]. This ensures the kurtosis contributes meaningfully to the sparsity calculation without introducing instability, making 0.1 a reasoned and effective choice.

Base Sparsity Ratio (α): The hyperparameter α functions as a global base sparsity ratio. It establishes a baseline sparse level for the entire model, providing a direct and interpretable control over the overall sparsification ratio. The kurtosis-based term then acts as a dynamic, layer-specific offset to this baseline.

In summary, Equation 7 implies the insight that the distribution of parameter importance reveals its tolerance to sparsifying. It begins with a global sparsity baseline (α), which is then intelligently adjusted for each layer using a logarithmically scaled measure of its importance concentration (kurtosis), thereby achieving a robust, adaptive, and effective sparsity allocation strategy. We leave other possible designs for adaptive determination of sparsity ratio for future work.

F Magnitude of B^{t-1}, A^{t-1} and $\Delta B^t, \Delta A^t$

In the Taylor expansion in Equation 11, the product term $\Delta B^t \Delta A^t$ is treated as a negligible higher-order term. To validate this assumption, we empirically analyze the magnitudes during the federated fine-tuning process. The average relative magnitude for matrix A, denoted as $\frac{\|\Delta A^t\|}{\|A^{t-1}\|}$, is less than 3%, while for matrix B, $\frac{\|\Delta B^t\|}{\|B^{t-1}\|}$, it is less than 10%. Their product constitutes a higher-order term that can be safely omitted. This conclusion is further reinforced by the observed convergence trend, where both relative ratios systematically decrease as the number of federated rounds increases.

G Performance of Base Models and Locally Trained Models

We test the performance of base models (Llama3.2-3B and Qwen2-7B) on our benchmarks. We also conduct local training on each client (e.g., Code indicates the model is locally trained on the client from the code domain) and test the locally trained models on each client on our benchmarks. The results are reported in Table 9.